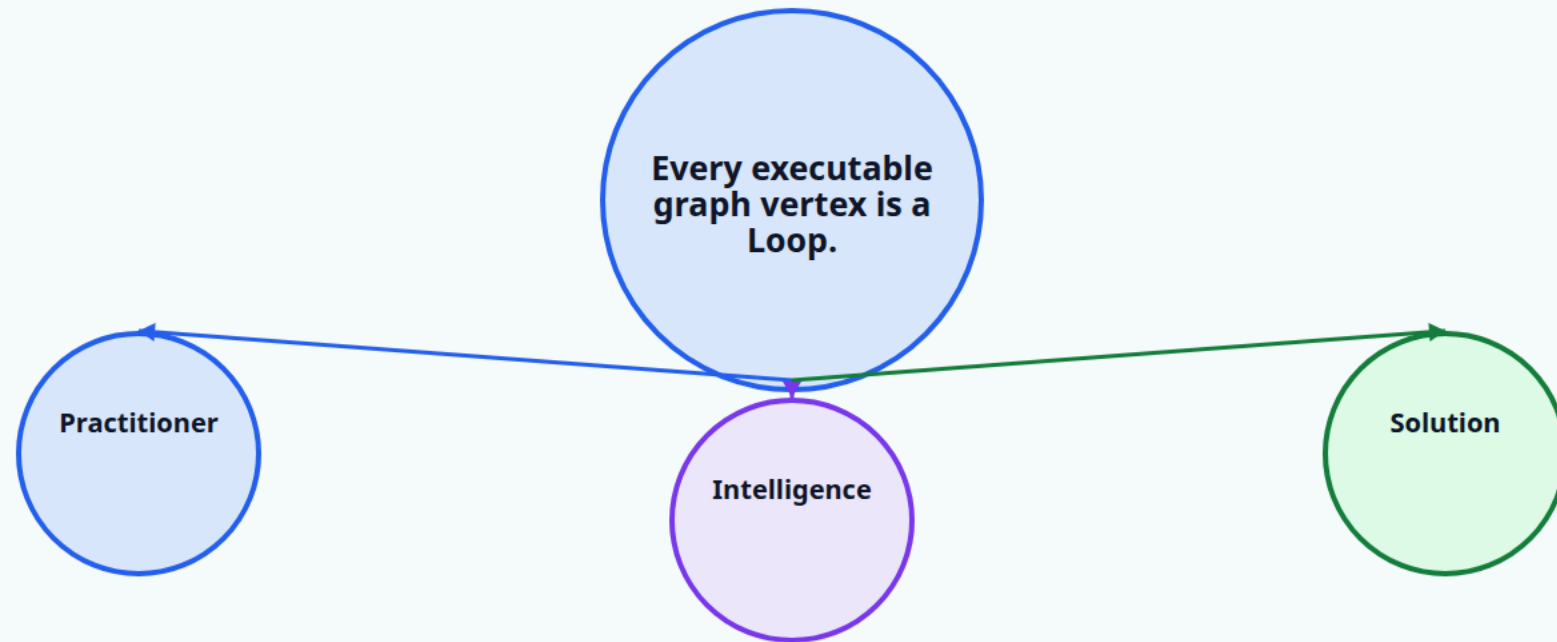


Loop Engine

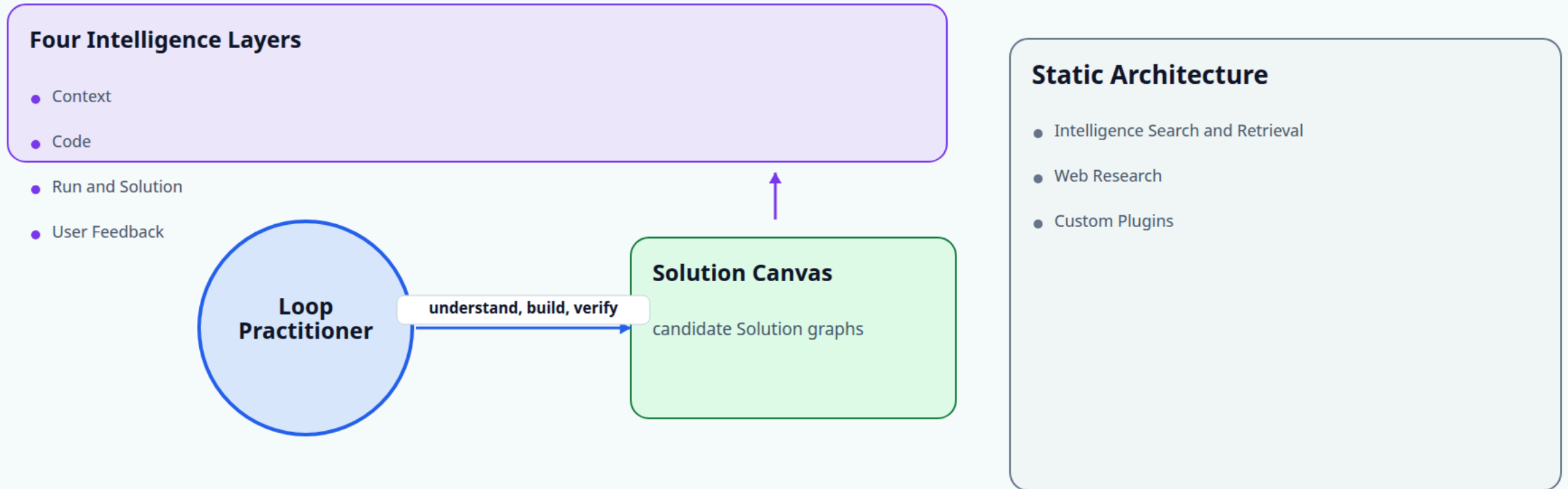
Loops are all you need.



This presentation separates the architecture contract from current implementation status.

One system, four clear parts

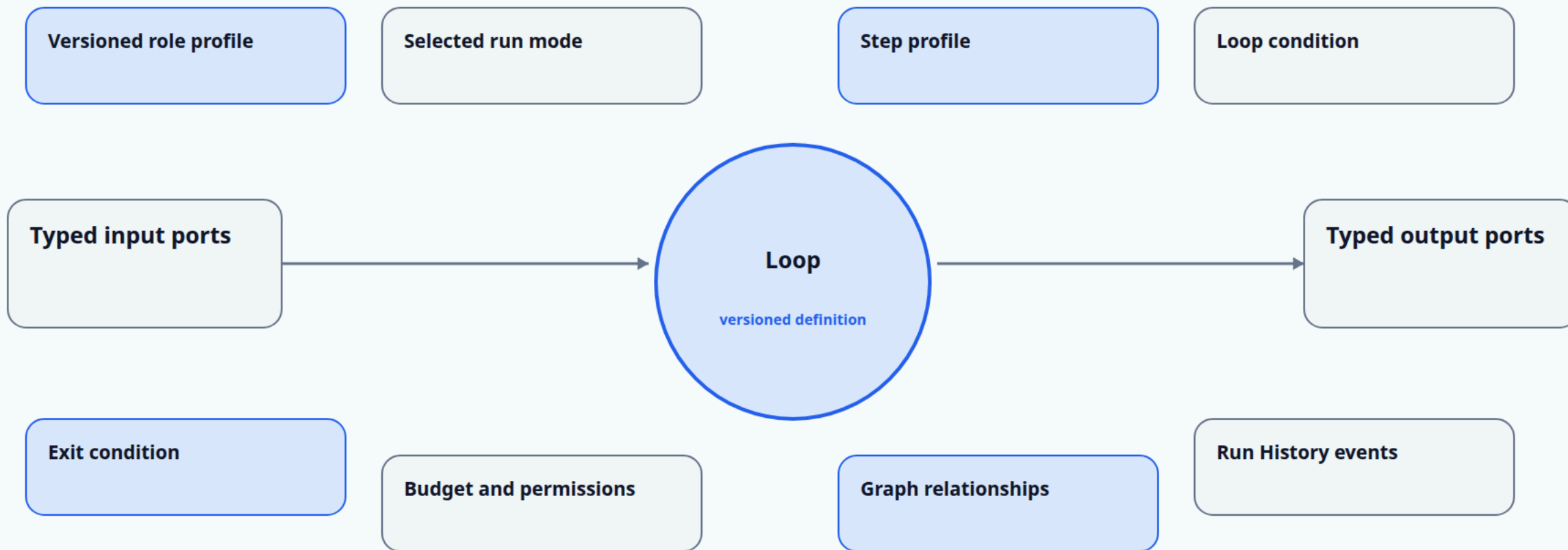
The Practitioner builds. The Canvas describes what runs. Intelligence and Static Architecture support the work.



Implemented. Established constructor calls remain visible as compatibility composition.

A Loop is one typed operational node

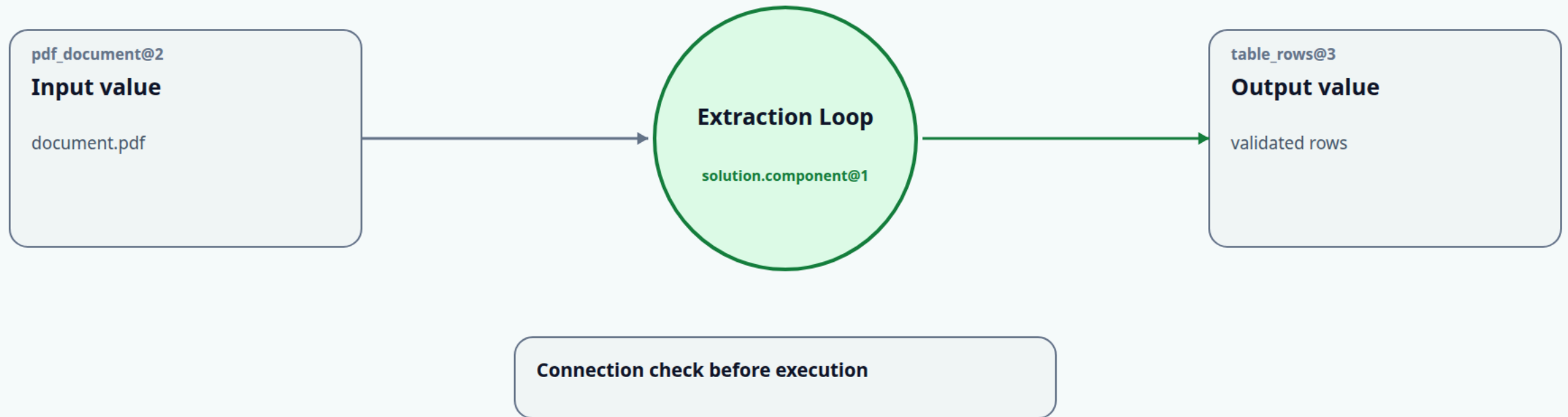
It has a stable definition, a run instance, and explicit graph relationships.



Implemented. One immutable definition binds profile, contract, modes, conditions, capabilities, version, and digest.

Input, work, validated output

A connection runs only when the source output matches the destination input.



Current port checks use role labels. Full value-schema enforcement remains planned.

Definition first, instance second

The definition states what may run. The instance records one actual run.

LoopDefinition

- ID, semantic version, digest
- role and exact profile
- typed ports and settings schema
- supported modes and effects



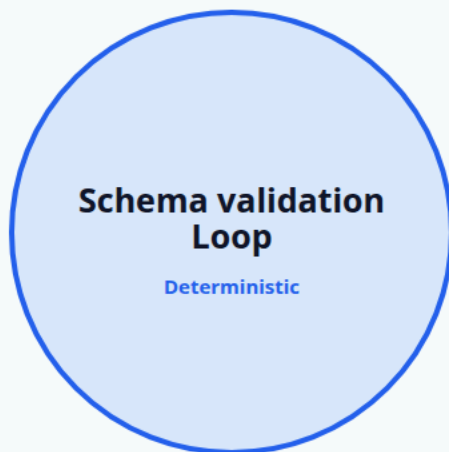
LoopInstance

- instance ID and definition reference
- selected mode and validated input
- status, counters, and relationships
- outputs and Run History references

Implemented. Every current Loop event includes its exact definition ID, version, and content digest.

Deterministic

Code, rules, retrieval, or calculation leads the work.



Code, rules, retrieval, or calculation leads.

Deterministic

- read typed input
- apply rules
- return typed result

Loop Engine controls permissions, budget, logging, and verification.

Example: validate a schema, calculate a metric, or retrieve an exact reference.

Hybrid

Code leads. A real model may resolve one bounded semantic step.



Code leads. A model may resolve one bounded semantic step.

Hybrid

- run deterministic extraction
- resolve bounded ambiguity
- validate the result

Model use does not grant file, network, secret, or spending permission.

Example: code extracts a table and a model resolves one ambiguous column heading.

Non-deterministic or LLM-driven

A real model leads the semantic work while Loop Engine keeps control.



A model leads the semantic work.

Non-deterministic or LLM-driven

- read bounded context
- deliberate with a real model
- return typed proposal

Tools, limits, approvals, history, and verification remain explicit.

Example: compare competing research plans, then return a structured recommendation for verification.

One choice per run

Each Loop instance selects one mode

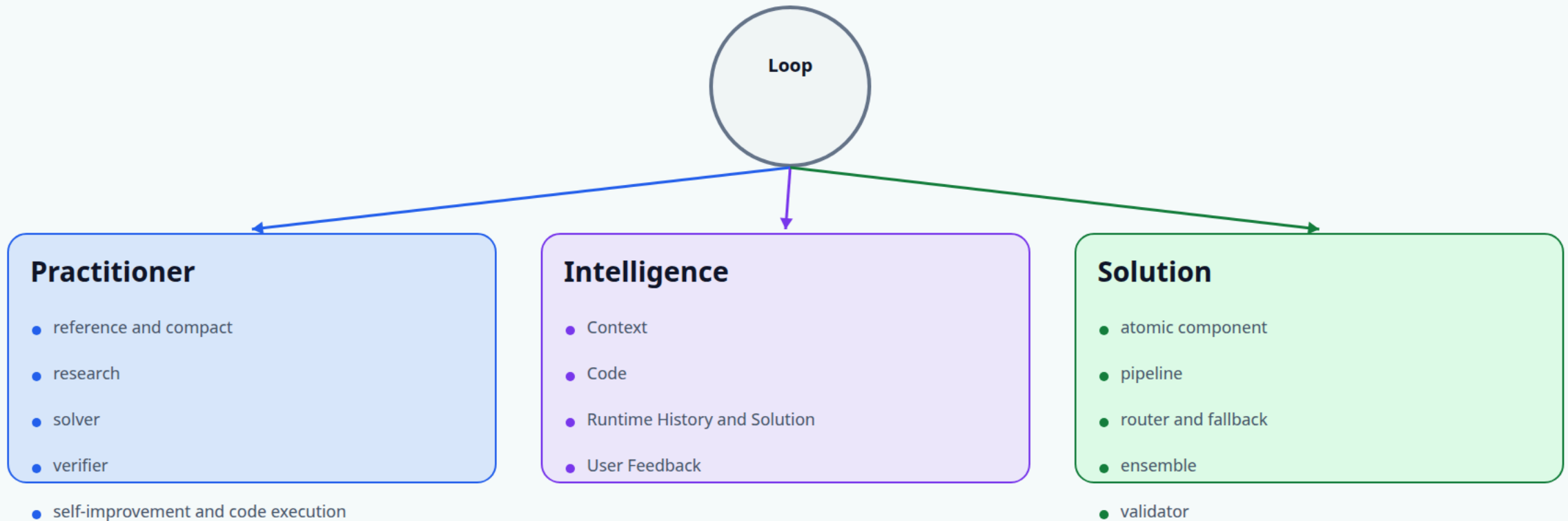
The graph and Canvas do not inherit one mode. Connected Loops may select different permitted modes.

Question	Deterministic	Hybrid	Non-deterministic
What leads?	code or rules	code	real model
Model involved?	no	only if needed	yes
Best fit	repeatable work	bounded ambiguity	open semantic work
Still controlled	effects and limits	effects and limits	effects and limits

Current Solution Canvas execution supports deterministic components. Other Solution modes fail preflight.

One Loop runtime, three role branches

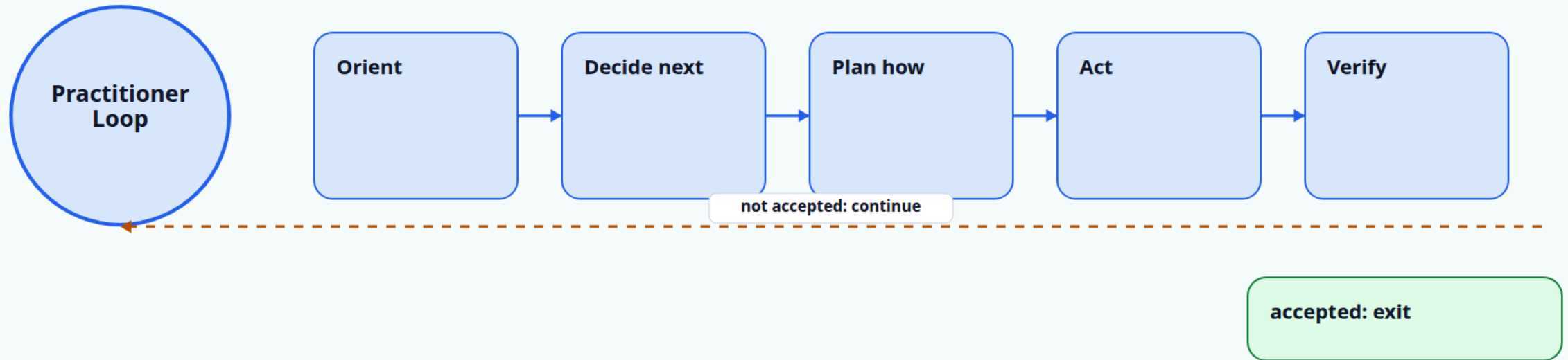
Role, mode, profile, category, and graph relationship stay separate.



Profiles are versioned behavior presets. They do not create another runtime class.

A Practitioner Loop decides what work is useful next

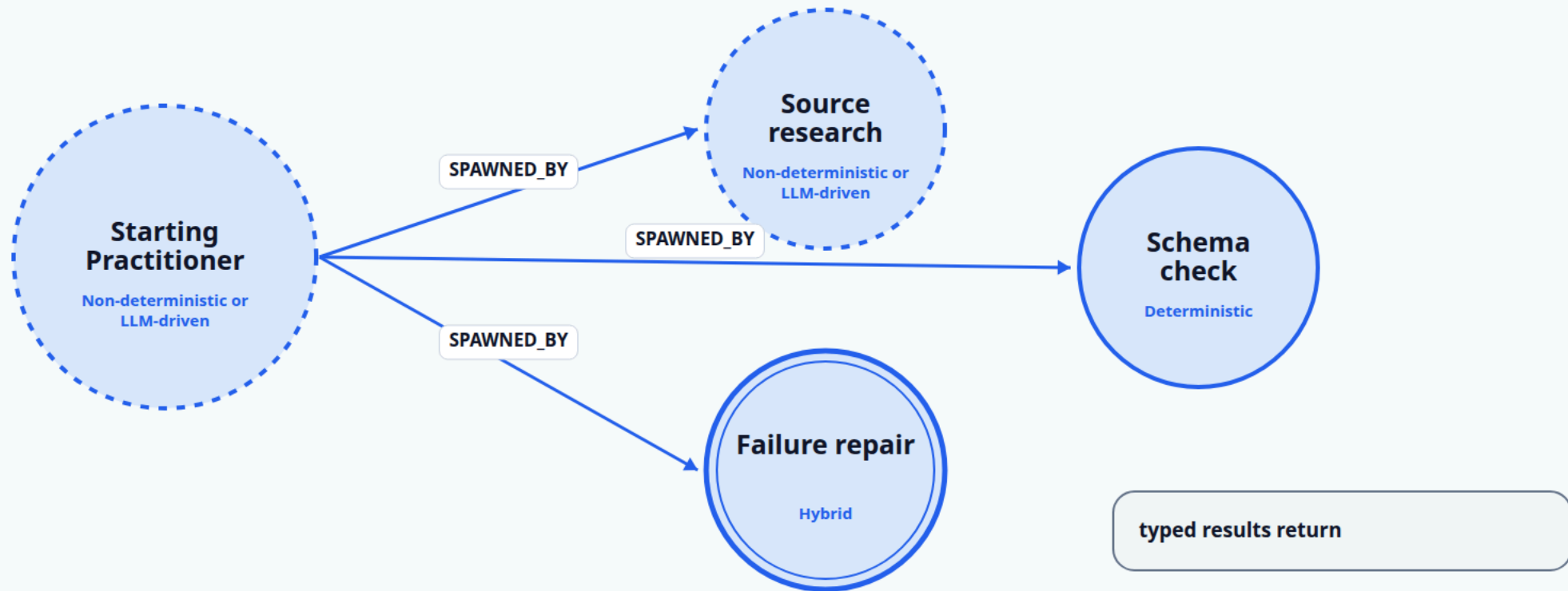
A simple presentation profile uses five steps. Runtime profiles may contain different steps.



The reference nine-step profile remains available. Custom profiles are valid when bounded and versioned.

A Practitioner can spawn focused Loops

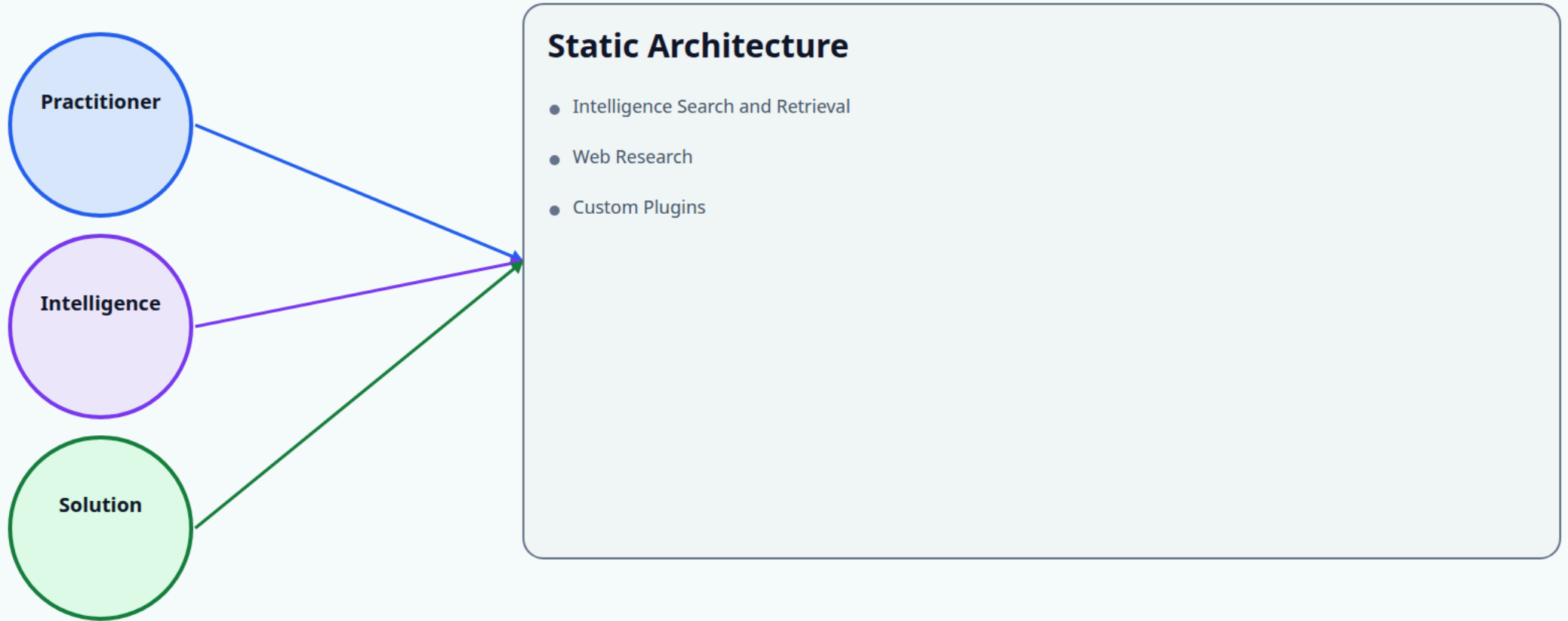
Each spawned Loop has its own profile, mode, typed contract, budget, and exit condition.



The typed lifecycle manager exists. Standard hybrid and non-deterministic spawned executors are not complete.

Every Loop should access the same three shared groups

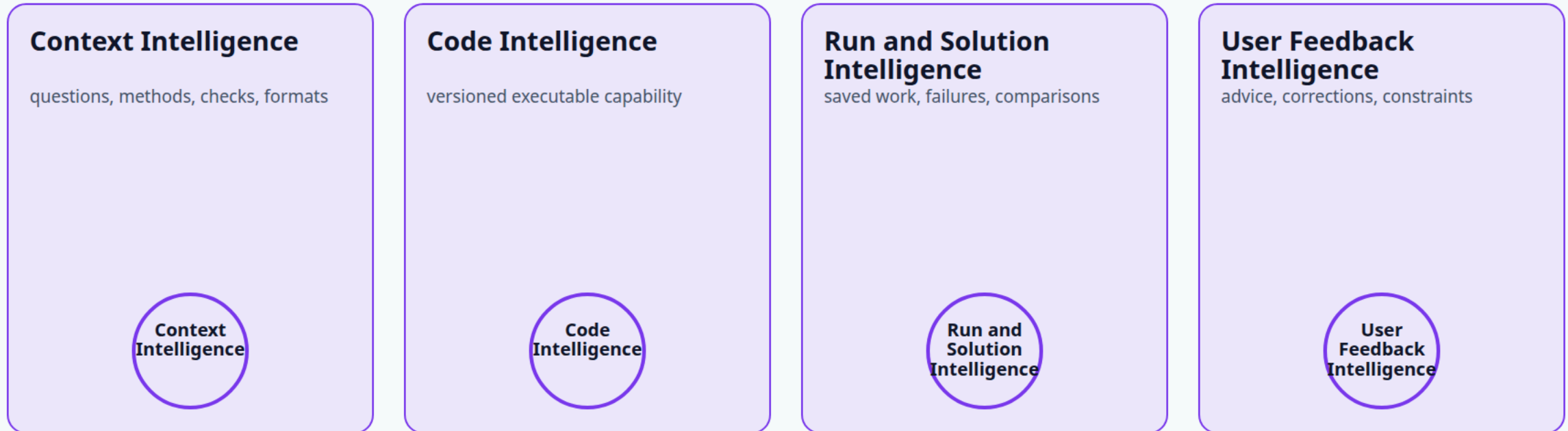
Typed service ports provide Intelligence Search and Retrieval, Web Research, and Custom Plugins.



Implemented. Each Loop carries a permission-limited `LoopRuntimeContext` with exactly these three public groups.

Intelligence is organized by what it contributes

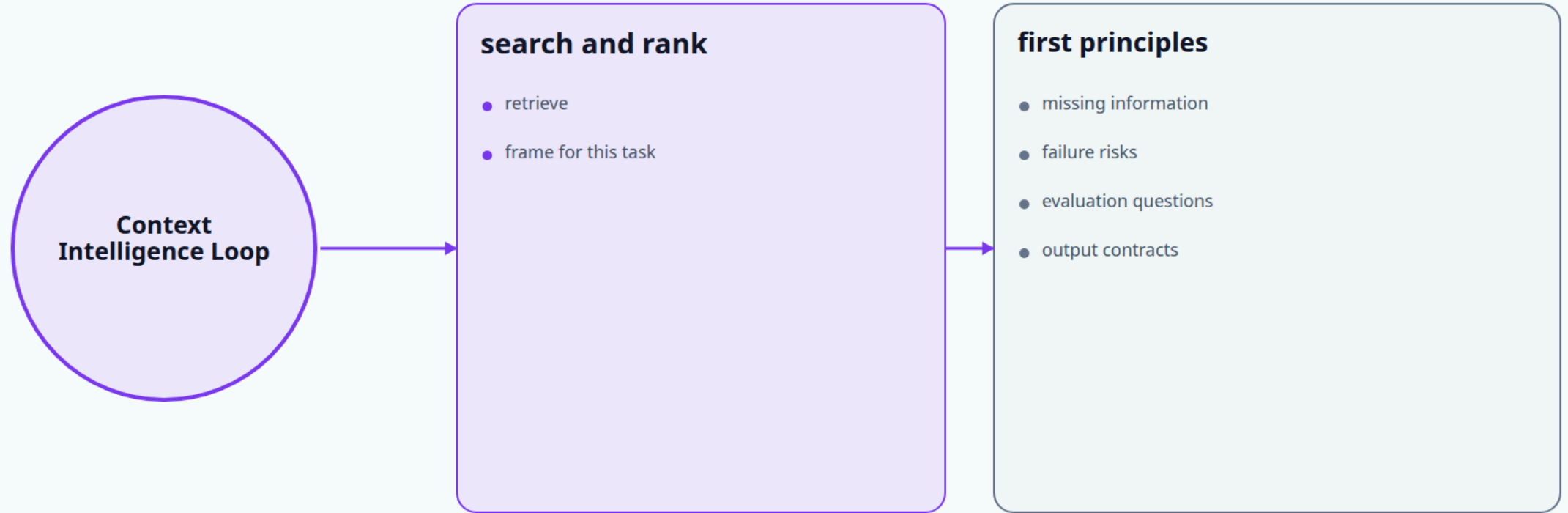
Every intelligence operation is a Loop. Passive content is materialized only after selection and permission checks.



Search, select, retrieve, frame, invoke, replay, compare, and interpret are Intelligence Loop profiles.

Context Intelligence

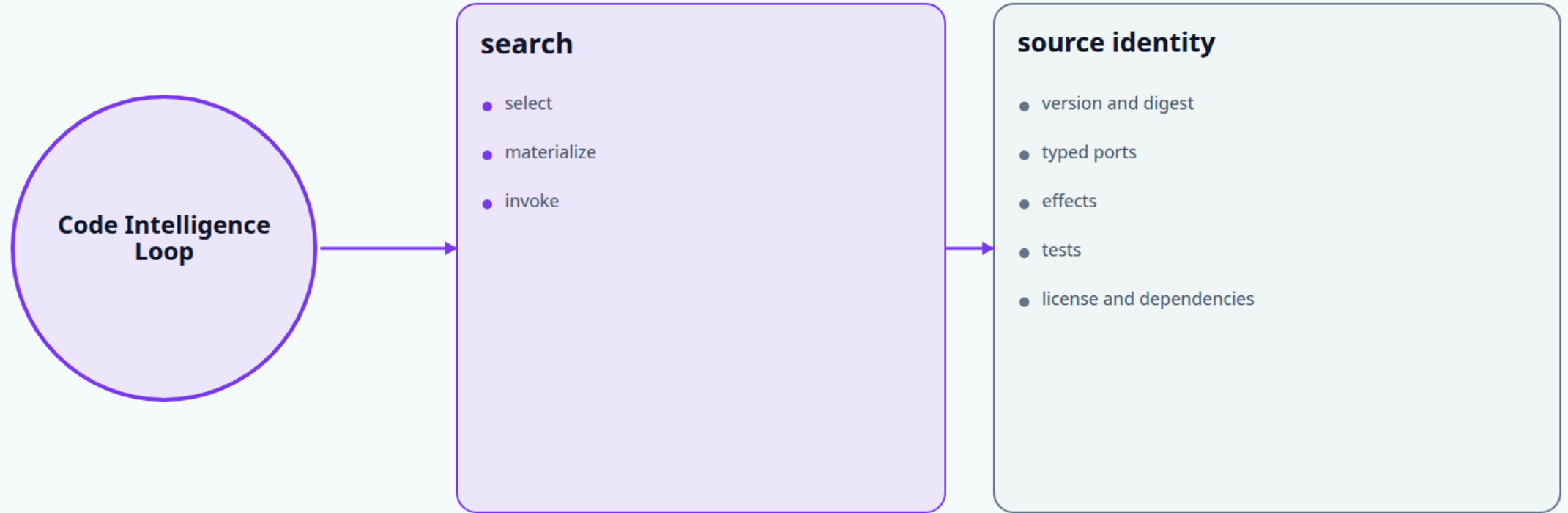
Reusable questions and ways of thinking help a Loop examine the task from several angles.



Imported prompts and skills remain candidates until independent review approves them.

Code Intelligence

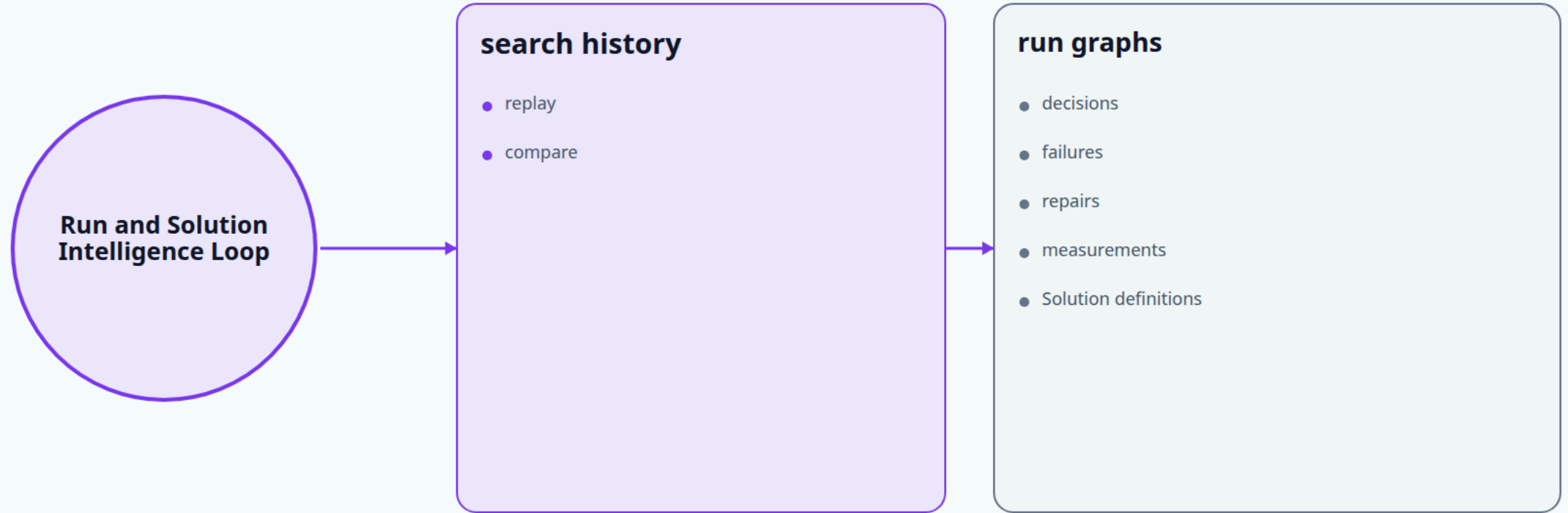
A callable, package, repository, or tool becomes usable only through an exact versioned contract.



Large bodies stay behind references until the selected Code Intelligence Loop needs them.

Runtime History and Solution Intelligence

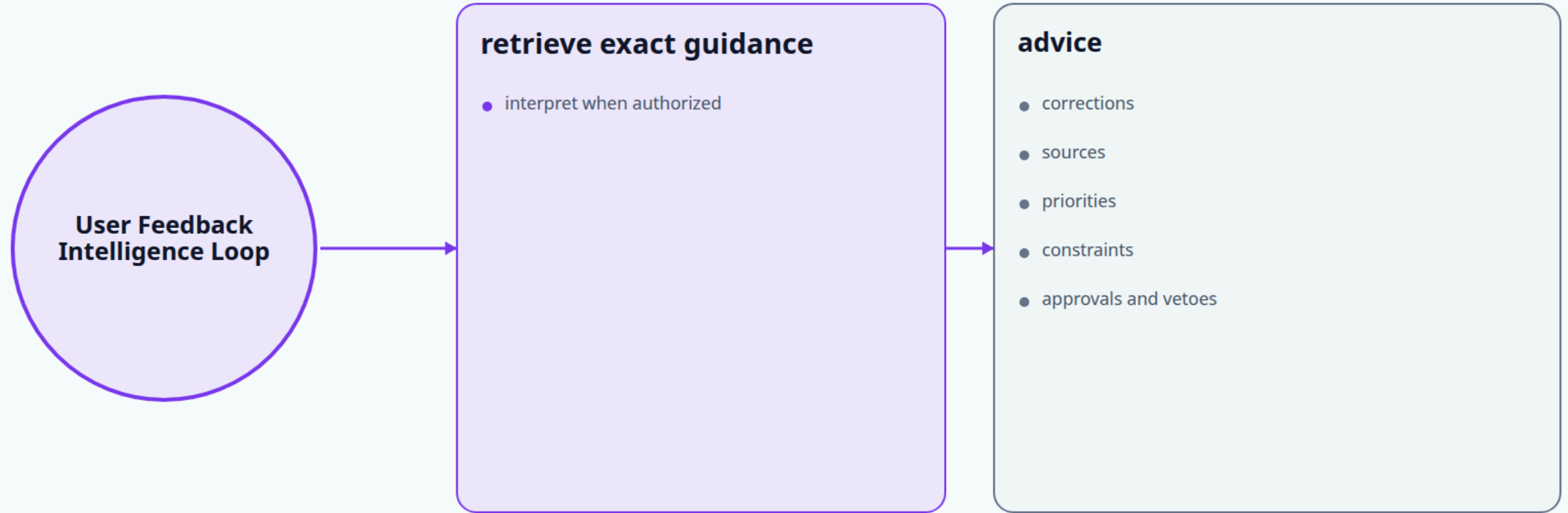
Saved work provides starting points, failures, repairs, measurements, and reusable Solution graphs.



The architecture uses one public name for this branch. Legacy storage names need migration readers.

User Feedback Intelligence

User advice, corrections, priorities, constraints, approvals, and vetoes remain explicit.



Interpretation never changes the original user record or grants a new permission.

The Solution Canvas keeps several working routes

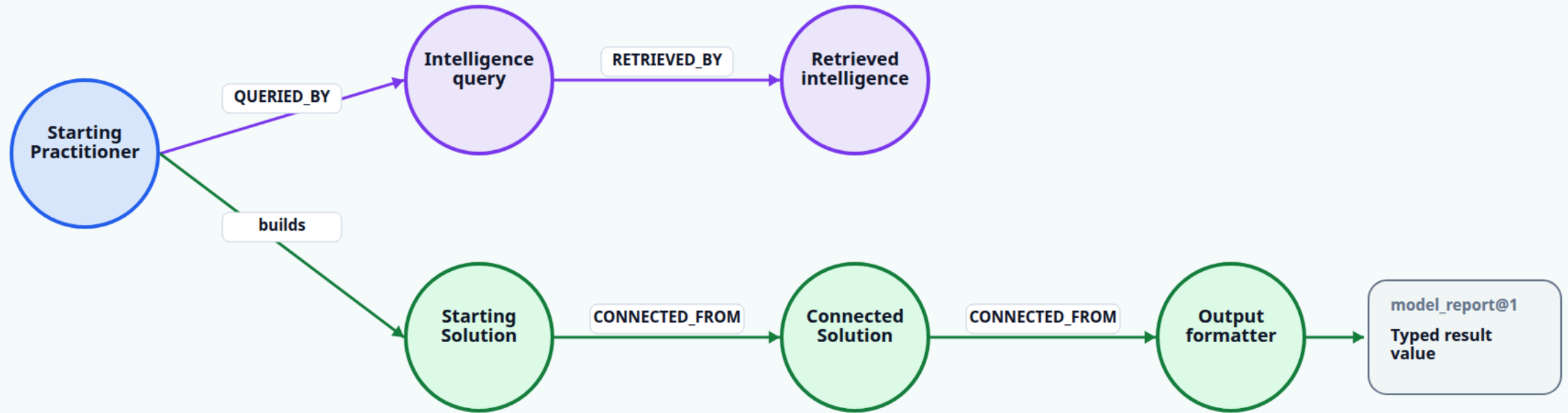
Each cell refers to a versioned Solution Loop. Typed edges connect the selected graph.

candidate	Capture	Extract	Normalize	Model	Verify
Route A	 browser	 table parser	 schema map	 boosted trees	 holdout
Route B	 HTTP client	 OCR parser	 rule map	 linear model	 holdout
Fallback	 saved files	 manual review	 repair	 baseline	 contract check

Implemented. SolutionSpec and Canvas now build or project one authoritative LoopGraphDefinition.

Loops connect into one typed directed acyclic graph

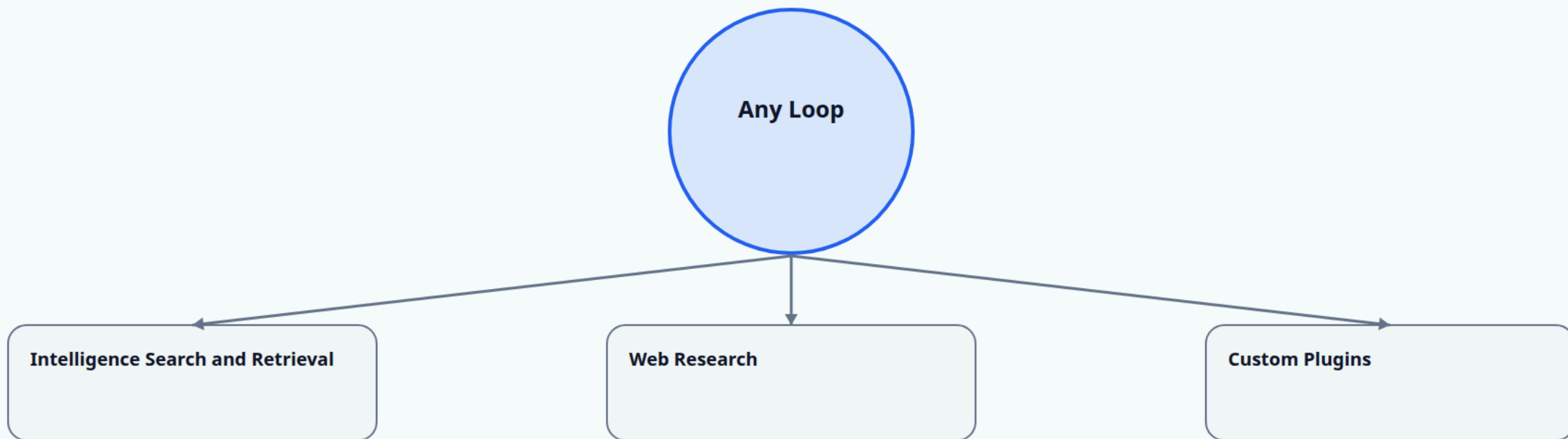
Repetition stays inside a Loop. Edges carry values and relationships, not hidden work.



Implemented. Every vertex resolves an exact versioned and digest-bound LoopDefinition.

Static Architecture has three public groups

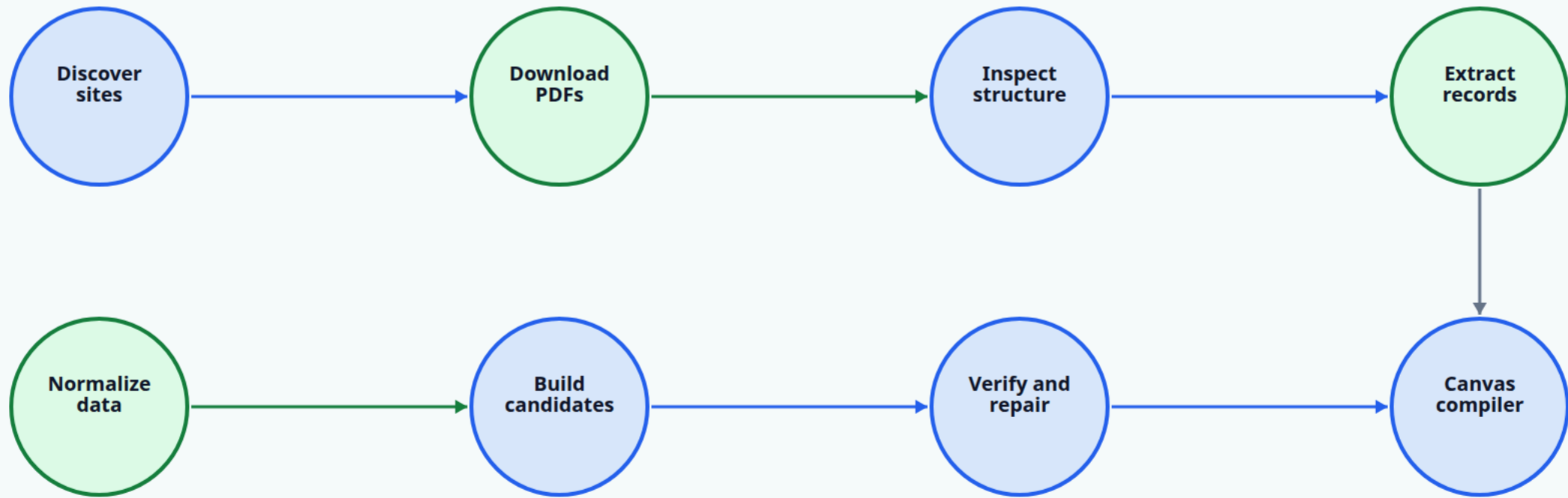
Every Loop can call these groups through typed, permission-limited service requests.



Other runtime mechanics stay internal and do not appear as peer architecture components.

From public websites to a tested model

The task combines web research, PDF extraction, data engineering, model building, and verification.



This is the proposed first end-to-end proof task. It is not a completed benchmark claim.

Find reliable sources and inspect every PDF

The Practitioner queries Intelligence before selecting search, download, and parsing capabilities.



Context and User Feedback questions

- Which sources are authoritative?
- Which dates and entities are in scope?
- Are tables, scans, and footnotes handled?

Different PDFs may require different extraction routes. One parser is not assumed to fit every document.

Build trusted data before building models

Selected Code Intelligence Loops extract, normalize, validate, and assemble model-ready records.



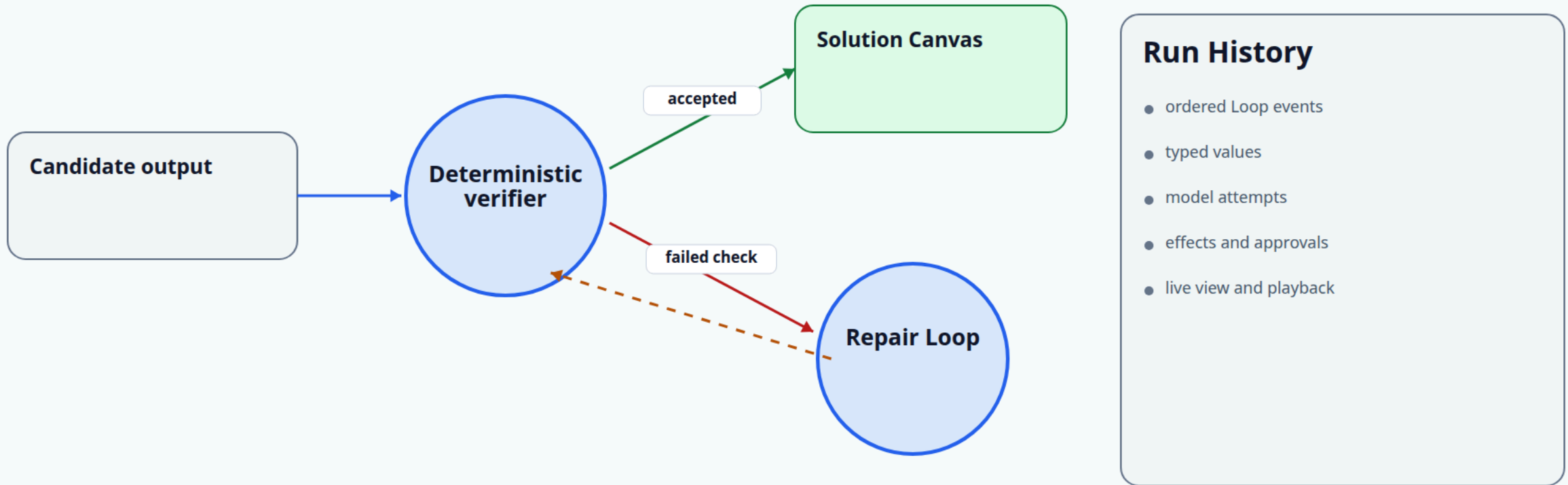
Intelligence questions

- What is missing or duplicated?
- Can future data leak into training?
- Which baseline and split prove useful performance?

Runtime History and Solution Intelligence supplies known failures, repair patterns, and evaluation geometry.

Verify, repair, compile, and record

A deterministic verifier accepts a candidate or starts a failure-specific repair Loop.



A report explains what ran. It does not turn one successful task into a general success rate.

Self-improvement uses the same Practitioner Loop

A Practitioner can review a bounded run population and stage candidates. Independent review remains separate.

Practitioner task profiles

- research
- build and test
- verify and repair
- self-improvement

Independent review decides promotion

Every executable graph vertex is a Loop.

Each Loop selects one mode and follows one versioned definition.

The Solution Canvas runs. The Run History explains what happened.

Remaining gaps: full value schemas and built-in hybrid or non-deterministic Solution executors.